

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor :	Vu Quang Dung
Student Name:	Nguyen Gia Bao
Student ID:	23010383
Email:	23010383@st.phenikaa-uni.edu.vn
Course class:	CSE703110-1-2-25(N01)

Ha Noi, March 2026

Contents

1. EXECUTIVE SUMMARY.....	2
1.1 Goal:.....	3
1.2 Final Achievement:.....	3
2. PROJECT REQUIREMENTS & GOALS.....	3
2.1 Functional Requirements (Core Features).....	3
2.2 Quality Attributes (Targeted Goals).....	3
3. ARCHITECTURAL DESIGN & IMPLEMENTATION.....	4
3.1 Technical Stack.....	4
3.2 Implementation of Core Logic (Operational Phases).....	4
4. USE CASE SPECIFICATIONS.....	5
4.1 System & Authentication (UC-01 to UC-03).....	5
4.2 Movie & Showtime Management (Admin) (UC-04 to UC-09).....	8
4.3 Browsing & Booking (User) (UC-10 to UC-15).....	13
4.4 Payment (UC-16).....	17
5. COMPLETED TASK DETAILS.....	18
5.1 Data Architecture Strategy.....	18
5.2 Database Schema Design.....	19
5.2.1 Movie Service Database (movies.db).....	19
5.2.2 Booking Service Database (booking.db).....	19
5.3 Data Integrity & Transaction Management.....	19
5.3.1 Seat Contention Handling (Concurrency Control).....	19
5.3.2 Atomic Transactions & Rollback.....	20
5.3.3 Event Publication (OrderPlacedEvent).....	20
5.4 Distributed Data Management.....	20
5.4.1 Cross-Service Cascade Delete Mechanism.....	20
5.4.2 Seat Logic & Dynamic Pricing.....	21

1. EXECUTIVE SUMMARY

1.1 Goal:

The primary objective of this project was to refactor a legacy Layered Monolith movie ticket booking application into a distributed Microservices Architecture.

Architectural Pattern: The system adopts a Microservices style utilizing Database-per-Service isolation and a Hybrid Communication Pattern (Synchronous REST APIs for reads and Asynchronous AMQP via RabbitMQ for writes).

1.2 Final Achievement:

The team successfully decomposed the application into independent containers (Gateway, Movie, Booking, Payment, Notification). The final system is orchestrated via Docker Compose, demonstrating high scalability, fault tolerance, and a clear separation of concerns, successfully addressing the limitations of the previous monolithic version.

2. PROJECT REQUIREMENTS & GOALS

2.1 Functional Requirements (Core Features)

- **Movie Management (CRUD):** Admin capabilities to Create, Read, Update, and Delete movies and showtimes.
- **Booking Transaction:** A complex workflow allowing users to select seats, lock them, and process payments.
- **Real-Time Messaging:** Event-driven notifications (Email/Tickets) and Invoice generation upon successful booking.
- **Search & Discovery:** High-performance movie searching for end-users.

2.2 Quality Attributes (Targeted Goals)

- **Scalability:** The ability to scale specific services (e.g., Booking Service) independently during high-traffic events.
- **Availability & Fault Tolerance:** The system must continue to accept bookings even if the Notification or Payment services are temporarily down (achieved via Message Queues).
- **Performance:**
 - *Read Performance:* Fast response times for movie listings using direct API calls.

- *Write Performance:* Non-blocking booking experience using asynchronous processing.

3. ARCHITECTURAL DESIGN & IMPLEMENTATION

3.1 Technical Stack

- Backend: Python 3.13 (Flask) for lightweight microservices.
- Frontend: Vanilla JavaScript (ES6+) and Bootstrap 5, bundled with Vite.
- Database: SQLite (Strict Database-per-Service implementation).
- Message Broker: RabbitMQ (Pika client) for event-driven communication.
- Infrastructure: Docker.

3.2 Implementation of Core Logic (Operational Phases)

Phase A: Infrastructure Bootstrap (Containerization)

- Orchestration: We use `docker-compose.yml` as the infrastructure blueprint. An internal Bridge Network is established to allow services to communicate via Hostnames (Service Discovery) rather than static IPs.
- Idempotent Initialization: Upon startup, each service checks for its specific database file (`movies.db`, `booking.db`). If absent, it automatically executes SQL schemas to create tables and seed initial data. This ensures strict data isolation with no cross-service foreign keys.

Phase B: Synchronous Operations (Read-Heavy Flows)

- API Gateway Pattern: The Flask Gateway (Port 5000) acts as the Single Entry Point. It intercepts Frontend requests, validates UUID-based Security Tokens stored in the database, and routes traffic to internal services (e.g., routing `/api/movies` to `movie-service:5001`).
- Direct Querying: For information retrieval, the Gateway forwards requests to the Movie Service, which queries its local DB and returns JSON directly. This minimizes latency for UI rendering.

Phase C: Asynchronous Operations (Write-Heavy Flows)

- Event-Driven Booking:
 1. Atomic Transaction: The Booking Service locks the seat in its local DB.

2. Producer: Immediately upon success, it publishes an *OrderPlacedEvent* directly to the *ticket_events* queue.
 3. Non-Blocking Response: The user receives a "Success" HTTP 201 response immediately, without waiting for email processing.
- Consumer Workers: The Notification and Payment services run as background workers. They consume messages from the queue to generate invoices and send emails via SMTP. If a worker crashes, the message remains in the queue (Persistence), ensuring no data loss.

Phase D: Technical Optimization

- Frontend Debounce: To prevent API overload, we implemented a custom useDebounce hook in Vanilla JavaScript. This limits search requests by only firing after the user stops typing for 500ms, reducing Gateway load by approximately 80%.

4. USE CASE SPECIFICATIONS

4.1 System & Authentication (UC-01 to UC-03)

UC-01: API Gateway Routing & Verification: Handles request routing, validates API Keys, and verifies User Tokens.

ID and Name:	UC-01: API Gateway Routing & Verification
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	The Gateway acts as the single entry point (Port 5000). It intercepts all requests, validates the API Key, verifies User Tokens (if present), and routes the request to the appropriate microservice.
Trigger:	Any HTTP Request sent to <u>http://localhost:5000</u>
Preconditions:	PRE-1: Gateway Service is running on Port 5000. PRE-2: User Service is running to verify tokens.
Postconditions:	POS-1: Request is successfully forwarded to the downstream service. POS-2: User identity headers (X-User-Email, X-User-Role) are injected.
Normal flow:	1. Client sends a request with Header peko-key.

	2. Gateway validates peko-key. 3. Gateway checks for Authorization: Bearer <token> header. 4. Gateway calls User Service to check token existence in <i>tokens</i> table. 5. User Service returns user profile and role. 6. Gateway injects user info into headers. 7. Gateway forwards request to the target service (Movie/Booking/Payment). 8. Gateway returns the response from the target service to the client.
Alternative Flows:	4a. Invalid Token: If User Service returns 401, Gateway denies request immediately.
Exceptions:	2a. Invalid API Key: Returns 401 Unauthorized. 5a. User Service Down: Returns 500 Internal Server Error.
Priority:	High

UC-02: User Registration: Allows new users to create an account.

ID and Name:	UC-02: User Registration
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Allows a new user to create an account in the system.
Trigger:	Guest sends: <i>POST /api/auth/register</i>
Preconditions:	PRE-1: Email must not already exist in users.db.
Postconditions:	POS-1: New user record created with role 'user'.
Normal flow:	1. User submits email and password. 2. Gateway forwards to User Service. 3. User Service hashes the password.

	4. User Service saves the record to users.db. 5. System returns success message.
Alternative Flows:	none
Exceptions:	4a. Email Exists: Returns 400 Bad Requests.
Priority:	High

UC-03: User Login: Authenticates users and issues Access Tokens (JWT).

ID and Name:	UC-03: User Login
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	Authenticates a user and issues a <i>Database-backed UUID Token</i> for accessing protected resources.
Trigger:	User sends <i>POST /api/auth/login</i>
Preconditions:	PRE-1: User/admin account must exist.
Postconditions:	POS-1: System returns a valid Access Token. POS-2: Client stores this token for future requests.
Normal flow:	1. User submits email and password. 2. Gateway forwards request to User Service. 3. User Service verifies email exists. 4. User Service validates password hash. 5. User Service generates a random <i>UUID Token</i> , stores it in the <i>tokens</i> table, and returns it. 6. System returns the Token to the user.
Alternative Flows:	none

Exceptions:	3a. User Not Found / Wrong Password: Returns 401 Unauthorized.
Priority:	High

4.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)

UC-04: Create Movie: Adds a new movie to the system.

ID and Name:	UC-04: Create Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Adds a new movie title to the system database.
Trigger:	Admin sends POST /api/movies
Preconditions:	PRE-1: User has 'admin' role. PRE-2: Movie ID must be unique.
Postconditions:	POS-1: New movie record is inserted into movies.db.
Normal flow:	1. Admin submits movie details (ID, Title, Genre, Duration). 2. Gateway checks Admin Token. 3. Movie Service checks if ID already exists. 4. Movie Service inserts data into movies table. 5. System returns "Movie Created" message.
Alternative Flows:	none
Exceptions:	3a. Duplicate ID: Returns 400 Bad Request.
Priority:	High

UC-05: Update Movie: Updates existing movie information.

ID and Name:	UC-05: Update Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Allows Admin to correct or update movie information (Title, Genre, Duration).
Trigger:	Admin sends PUT /api/movies/{id}
Preconditions:	PRE-1: Movie exists. PRE-2: User has 'admin' role.
Postconditions:	POS-1: Movie details updated in movies.db.
Normal flow:	1. Admin submits new details (e.g., fix typo in Title). 2. Gateway verifies Admin Token. 3. Movie Service updates the record in database. 4. System returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-06: Cascade Delete Movie: Deletes a movie along with its showtimes and triggers automatic ticket refunds.

ID and Name:	UC-06: Cascade Delete Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Deletes a movie and automatically deletes all associated showtimes and refunds all booked tickets.
Trigger:	Admin sends DELETE /api/movies/{id}
Preconditions:	PRE-1: User must have the 'admin' role.

	PRE-2: The Movie ID must exist.
Postconditions:	POS-1: Movie, Showtimes, and Bookings are deleted. POS-2: REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion. 2. Movie Service finds all associated showtimes. 3. Loop: Trigger Refund Process for each showtime. 4. Delete all showtimes. 5. Delete the movie record. 6. Returns the count of refunded tickets.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-07: Create Showtime: Schedules a movie and automatically generates available seats.

ID and Name:	UC-07: Create Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Schedule a movie and generate seats.
Trigger:	Admin sends POST /api/showtimes
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist. PRE-3: Time is valid.

	PRE-4: The total number of seats must not exceed 64.
Postconditions:	POS-1: Showtime saved in movies.db. POS-2: Seats created in booking.db.
Normal flow:	1. Admin sends time and price. 2. Movie Service saves showtime. 3. Movie Service calls Booking Service (/internal/seats) to generate seats. 4. Returns success.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-08: Update Showtime & Auto-Refund: Updates showtime details (triggers refunds if the price changes).

ID and Name:	UC-08: Update Showtime & Auto-Refund
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Update showtime. If price changes, existing bookings are automatically cancelled and refunded.
Trigger:	Admin sends PUT /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Update successful.

	POS-2: (If price changed) Tickets cancelled, REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin changes price. 2. Movie Service detects price change. 3. Calls Booking Service to cancel all tickets. 4. Publishes REFUND_NOTIFICATION to RabbitMQ. 5. Updates new price in DB.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-09: Delete Showtime: Deletes a specific showtime and refunds any sold tickets.

ID and Name:	UC-09: Delete Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Delete a specific showtime and refund sold tickets.
Trigger:	Admin sends DELETE /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Showtime deleted. POS-2: REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion.

	2. Gateway verifies Admin Role. 3. Trigger Refund Process. 4. Delete record in DB. 5. Returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	High

4.3 Browsing & Booking (User) (UC-10 to UC-15)

UC-10: Search Movies: Lists all movies or searches for specific ones by title/ID.

ID and Name:	UC-10: Search Movies
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	List all movies or search by title or show one movies by id
Trigger:	GET/api/movies(with?query=...) GET/api/movies/{id}
Preconditions:	PRE-1: Movie Service is running. PRE-2: Database has movie records (optional).
Postconditions:	POS-1: Returns list of movies matching query. POS-2: No database modification.
Normal flow:	1. User sends query. 2. Service performs LIKE search. 3. Returns list.

Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-11: View Showtimes Schedule: Displays available showtimes and pricing.

ID and Name:	UC-11: View Showtimes Schedule
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	View list of available showtimes and prices.
Trigger:	GET /api/showtimes
Preconditions:	PRE-1: Movie Service is running.
Postconditions:	POS-1: Returns list of showtimes. POS-2: No database modification.
Normal flow:	1. User requests schedule. 2. System returns list of showtimes.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-12: Book Ticket: Reserves a seat and publishes an order event to the system.

ID and Name:	UC-12: Book Ticket
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User reserves a seat. Price is fetched dynamically.

Trigger:	POST /api/bookings
Preconditions:	PRE-1: User must be logged in. PRE-2: Seat must be 'available'.
Postconditions:	POS-1: Booking created. POS-2: The OrderPlacedEvent event is sent to the ticket_events queue.
Normal flow:	1. User selects seat. 2. Service checks availability. 3. Calls Movie Service to fetch price. 4. Creates booking and updates seat status. 5. Sends event to RabbitMQ.
Alternative Flows:	none
Exceptions:	2a. Seat Taken: Returns 400 Bad Request.
Priority:	High

UC-13: View Bookings: Allows users to view their personal booking history.

ID and Name:	UC-13: View Bookings
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View history of bookings made by the user.
Trigger:	GET /api/bookings
Preconditions:	PRE-1: User must be logged in.
Postconditions:	POS-1: Returns list of bookings for that user.
Normal flow:	1. Gateway extracts email from Token.

	2. Service filters bookings by email. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-14: View Booking Detail: Shows specific details for a single ticket.

ID and Name:	UC-14: View Booking Detail
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View specific details of a single ticket.
Trigger:	GET /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Returns ticket details.
Normal flow:	1. User requests ID. 2. System verifies ownership. 3. Returns details.
Alternative Flows:	none
Exceptions:	2a. Not Owner: Returns 403 Forbidden.
Priority:	High

UC-15: Cancel Booking (Manual): Allows users to manually cancel their own bookings.

ID and Name:	UC-15: Cancel Booking (Manual)
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User cancels their own ticket. Note: Manual cancellation does NOT trigger a refund event in the current implementation.
Trigger:	DELETE /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Booking deleted. POS-2: Seat becomes available.
Normal flow:	1. User requests cancellation. 2. System verifies ownership. 3. Deletes booking. 4. Frees up seat.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

4.4 Payment (UC-16)

UC-16: Process Payment: Processes mock transactions, logs payments, and generates invoices.

ID and Name:	UC-16: Process Payment
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Pay for a ticket and generate an invoice.
Trigger:	POST /api/payments

Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: Valid Booking ID exists.
Postconditions:	POS-1: Payment logged. POS-2: INVOICE_PRINT event sent.
Normal flow:	1. User submits Booking ID. 2. Gateway verifies Token and routes to Payment Service. 3. Payment Service calls Booking Service (Internal API) to verify the booking exists and get the correct amount. 4. Payment Service processes the mock transaction. 5. Payment Service publishes InvoiceGenerated event to RabbitMQ. 6. System returns "Payment Successful" message to User.
Alternative Flows:	none
Exceptions:	3a. Booking Not Found: Payment Service returns 404. 3b. Already Paid: Payment Service returns 400 Bad Request.
Priority:	High

5. COMPLETED TASK DETAILS

5.1 Data Architecture Strategy

The project applies the Database-per-Service model, a core standard in Microservices architecture. Instead of using a large shared database, each service owns and manages its own dedicated database to ensure decoupling (reduced dependencies) and scalability.

Movie Service: Manages movie catalogs, screening rooms, and showtimes (movies.db).

Booking Service: Manages ticket booking transactions, seat status, and tickets (booking.db).

Technology used: SQLite, suitable for development and demo environments and easily migratable to PostgreSQL/MySQL in production.

5.2 Database Schema Design

5.2.1 Movie Service Database (movies.db)

This database serves as the master data source for the entire system.

movies table: Stores static movie information.

Key fields: id (PK), title, genre, duration (minutes), release_date.

rooms table: Defines the physical structure of screening rooms.

Key fields: id (PK), name, rows (number of rows), seats_per_row.

showtimes table: Links movies and screening rooms by time.

Key fields: id (PK), movie_id (FK), room_id (FK), start_time, end_time, price (base price).

Note: Although foreign keys are defined, deletion-related referential integrity (Cascade Delete) is handled at the application layer to support complex refund business logic.

5.2.2 Booking Service Database (booking.db)

This database focuses on transaction integrity and inventory state management.

seats table: Manages the status of each seat for a given showtime.

Key fields: id (PK), showtime_id, seat_number, status (available / booked), seat_type (STANDARD / VIP / COUPLE), price_surcharge.

Constraint: UNIQUE(showtime_id, seat_number) to prevent duplicate seats.

bookings table: Stores ticket booking transactions.

Key fields: id (PK), showtime_id, seat_number, customer_email, amount, status (default: PENDING_PAYMENT).

5.3 Data Integrity & Transaction Management

5.3.1 Seat Contention Handling (Concurrency Control)

The main challenge in a ticket booking system is race conditions, where multiple users attempt to book the same seat simultaneously. This issue is handled using Optimistic Locking through atomic SQL update statements.

Logic: A seat can be updated to booked if and only if its current status is available.

Result validation: After execution, the system checks cursor.rowcount.

If rowcount > 0: The seat is successfully booked → proceed with booking creation.

If rowcount == 0: The seat has already been booked by another user → rollback and return an error to the user.

5.3.2 Atomic Transactions & Rollback

The ticket booking workflow (book_ticket) is wrapped in a transaction to ensure consistency:

Validation: Verify the availability of all selected seats before processing.

Lock & Insert: Perform atomic updates for each seat and insert records into the bookings table.

Best-Effort Rollback: If a multi-seat booking fails midway (e.g., the first seat succeeds but the second fails), the system triggers a manual rollback by deleting created bookings and restoring seat statuses to available.

5.3.3 Event Publication (OrderPlacedEvent)

After a successful commit to the database, the Booking Service acts as a Producer. It creates a Message containing the minimum necessary information (Payload), including BookingID, UserID, Amount, and ShowtimeInfo. This Message is pushed directly to the RabbitMQ ticket_events queue. By decoupling this process, the user receives a "Success" HTTP 201 response immediately, without waiting for the email processing, ensuring a Non-blocking I/O experience.

5.4 Distributed Data Management

5.4.1 Cross-Service Cascade Delete Mechanism

Because data is stored in separate databases, deleting a movie or showtime requires coordinated processing across services instead of relying on SQL ON DELETE CASCADE.

Movie deletion workflow (DELETE /api/movies/{id}):

Retrieval: The Movie Service retrieves all showtimes associated with the movie.

Cancellation Request (Synchronous): Calls an internal API of the Booking Service (DELETE /api/internal/bookings/...) to cancel related bookings.

Refund Processing: The Booking Service deletes bookings and returns a list of affected customers.

Notification (Asynchronous): The Movie Service publishes a REFUND_NOTIFICATION event to RabbitMQ to notify customers.

Cleanup: After all business logic is completed, the Movie Service deletes the related showtimes and movie records from its own database.

5.4.2 Seat Logic & Dynamic Pricing

The system does not store a fixed seat layout. Instead, it uses Lazy Initialization combined with a seat generation algorithm.

Initialization: Seat data is generated and inserted into the seats table only when a seat query is made for a showtime for the first time, based on room configuration (number of rows and seats per row).

Automatic seat classification:

Rows A–D: STANDARD seats (no surcharge).

Rows E–H (Seats 4–8): VIP seats (20,000 VND surcharge).

Row I: COUPLE seats (50,000 VND surcharge).